Multiagente BOTI - Orquestador + Agente Base

CABA – IMC

DOCUMENTACIÓN TÉCNICA DEL PROYECTO

Desarrollado por

Sobre este documento: Versión 1.0, 18/12/2025.

PI DATA STRATEGY & CONSULTING

Copyright © Pi Data Strategy & Consulting 2020

Santa Rosa 320 Piso 5 A – Teléfono +54(351)5693075
Email: info@piconsulting.com.ar
www.piconsulting.com.ar

Índice

Contenido

Índice 2
Control de versiones 3
Registro de equipo de desarrollo de proyecto 4
Introducción 5
Información general del proyecto 6
Información técnica del proyecto 7
- Solución 7
- Orquestador 11
- Agente Base 19
- Ingesta de la Base de Conocimientos 22
- Arquitectura 23
- Seguridad 25
- Pruebas 26
- Despliegue 31
- Monitoreo 33

Gestión del proyecto 37
Recomendaciones y mantenimiento del modelo 40

Control de versiones

Versión	Descripción del cambio	Aprobado por	Fecha
0.1	Integración y control de las distintas secciones del documento	Dario Peralta	12/01/26

Historial de contribuciones

Descripción del cambio	Aprobado por	Fecha
Creación del documento y definición inicial del alcance (Orquestador y Agente Base).	Matías Bravo	15/12/25
Redacción de la información técnica: sección del Orquestador	Federico Ribetto	19/12/25
Redacción de la información técnica: sección del Agente Base..	Matías Bravo	22/12/25
Ampliación del detalle técnico del orquestador/Agente Base	Joaquín Marsan	23/12/25
Redacción de la información técnica: Pruebas y monitoreo	Norberto Luna	29/12/25
Redacción de la sección Gestión del Proyecto	Vilma Glenzel	12/01/26
Integración y control de las distintas secciones del documento	Dario Peralta	12/01/26
Integración de la estructura de datos guardada	Joaquín Marsan	13/01/26

Registro de equipo de desarrollo de proyecto

Nombre	Rol	Empresa	Email
Darío Peralta	Delivery Manager	PI Data	dperalta@piconsulting.com.ar
Vilma Glenzel	Project Manager	PI Data	vglenzel@piconsulting.com.ar
Norberto Luna	QA Analyst	PI Data	norberto.luna@piconsulting.com.ar
Joaquín Marsan	Backend Developer	PI Data	joaquin.marsan@piconsulting.com.ar
Federico Ribetto	AI Engineer	PI Data	federico.ribetto@piconsulting.com.ar
Matías Bravo	AI Engineer	PI Data	matias.bravo@piconsulting.com.ar

Introducción

La **Documentación Técnica del Proyecto Multiagente BOTI - Orquestador + Agente Base** es un documento técnico-funcional diseñado para detallar la arquitectura, despliegue y operación de la nueva solución conversacional implementada para el **Gobierno de la Ciudad de Buenos Aires (CABA)**.

Este documento tiene como objetivos específicos:

- **Visión Arquitectónica:** Proporcionar a los administradores y arquitectos de software una descripción profunda de la infraestructura desplegada en **Microsoft Azure** (Container Apps, Functions, AI Foundry), detallando cómo interactúan los componentes en un entorno *serverless*.
- **Entendimiento Lógico:** Explicar a los equipos de desarrollo la lógica de negocio implementada a través del orquestador y los agentes basados en **LangGraph**, incluyendo los flujos de decisión, clasificación de intenciones y generación de respuestas (RAG).
- **Soporte y Mantenimiento:** Servir de guía de referencia para el equipo de soporte técnico especializado (Nivel 2) respecto a los procesos de ingesta de datos (ETL), vectores de seguridad, manejo de *guardrails* y monitoreo de la salud del sistema.

El alcance de este documento cubre desde la recepción del mensaje del vecino hasta la generación de la respuesta mediante Inteligencia Artificial Generativa, abarcando la integración con fuentes de datos oficiales (Drupal) y la gestión del ciclo de vida de los datos.

Información general del proyecto

Objetivos

Los objetivos propuestos para el proyecto son:

- Acercar al vecino una forma más práctica de interactuar con Boti.
- Evolucionar la experiencia de usuario dejando de lado los menús estáticos.
- Permitir una conversación dinámica para que el usuario obtenga la información que necesite de manera natural.

Contexto organizacional

Para el **Gobierno de la Ciudad de Buenos Aires (CABA)**, este proyecto busca impactar directamente en la calidad de atención y servicio al vecino, optimizando los canales de comunicación actuales mediante el uso de Inteligencia Artificial.

Descripción general del producto o resultado del proyecto

El producto obtenido consiste en el desarrollo de una arquitectura **Multi-Agente en Azure**, compuesta por:

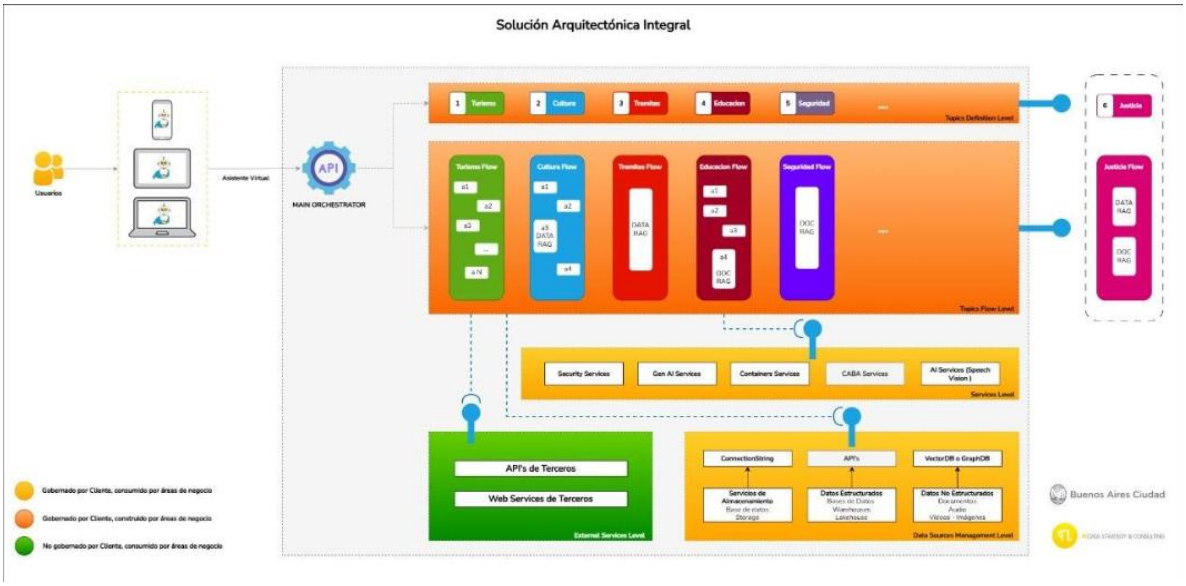
- Un **Orquestador** encargado de gestionar las interacciones.
- Un **Agente Base** diseñado para ser replicado y adaptado por cada temática específica que se requiera integrar.

Información técnica del proyecto

Solución

Proponemos una plataforma conversacional multi-agente gobernada por un Orchestrator que busca precisión, seguridad y experiencia homogénea de cara al ciudadano. La orquestación decide dinámicamente qué agente especializado debe intervenir según la intención detectada y consolida una única respuesta con tono unificado (personalidad). Cada agente fundamenta sus respuestas contra fuentes curadas en Azure AI Search y modelos de Azure OpenAI, reduciendo al mínimo la alucinación y elevando la confiabilidad. La Plantilla de Agente Base estandariza despliegues, observabilidad y cumplimiento, facilitando la evolución del catálogo de agentes sin impacto en el canal conversacional.

El usuario interactúa a través del canal conversacional. Un Orchestrator central clasifica la intención, selecciona el agente más adecuado (Accesibilidad, Trámites, Turismo) y coordina su ejecución. Cada agente utiliza un modelo de Azure OpenAI y una base vectorial en Azure AI Search como información base (RAG) y responde con conocimiento verificado. El Orchestrator aplica guardrails, grounding y unifica la personalidad (tono, estilo, cortesía) antes de devolver una única respuesta coherente al usuario.



Descripción

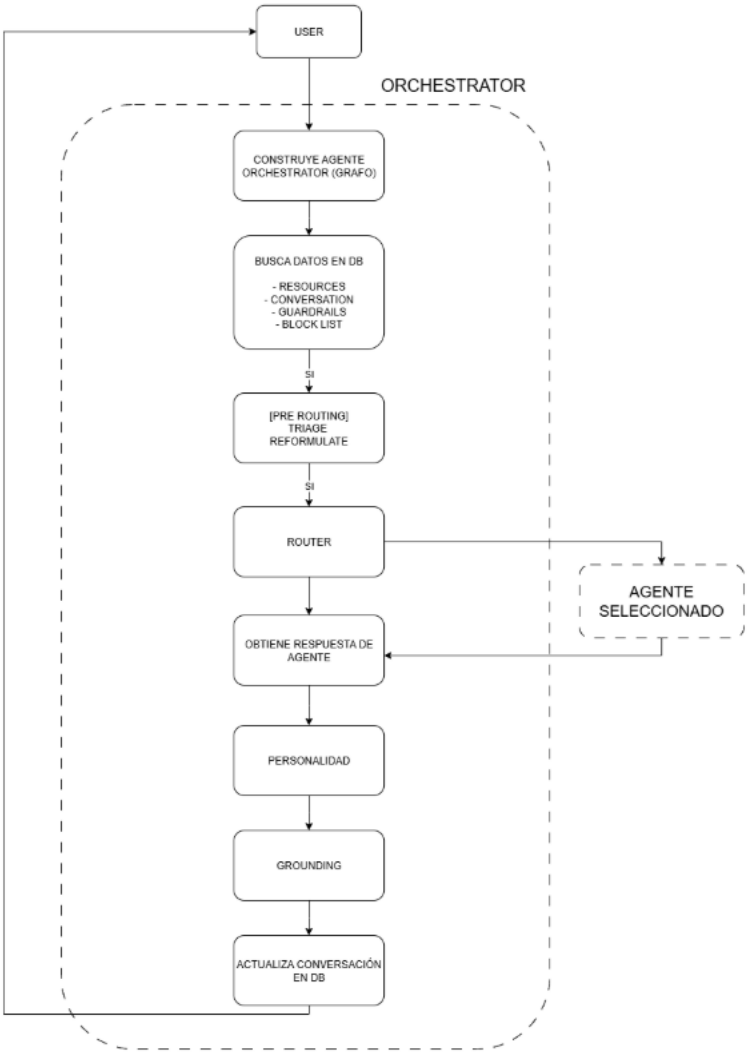
- Canal conversacional: punto de entrada de consultas de los usuarios.
- Orchestrator (componente central):
  - Enrutamiento por intención: determina qué agente(s) deben intervenir con base en clasificación de intención y confianza.
  - Persistencia: Gestiona el historial de conversaciones en la base de datos.
  - Coordinación: invoca a un agente y consolida sus respuestas.

- Unificación de personalidad: aplica reglas de tono y estilo institucional para que toda respuesta sea consistente, más allá del agente que la originó.
- Grounding & guardrails: valida, refuerza y filtra contenido (moderación, cumplimiento, políticas) antes de responder.
- Observabilidad: registra telemetría, latencias, fuentes citadas y resultados para mejora continua.

- Agentes especializados (Accesibilidad, Trámites, Turismo, etc.):
- Plantilla de Agente Base: todos comparten el mismo esqueleto técnico y operan bajo estándares comunes (seguridad, logging, métricas, manejo de errores).
- Capacidades:
  - Acceso a Azure OpenAI (razonamiento y generación controlada).
  - Vector Store en Azure AI Search para recuperación semántica de conocimiento (RAG).
  - Lógica de dominio para refinar consultas, citar evidencia y estructurar salidas.
  - Fuentes de conocimiento: índices vectoriales por dominio (políticas, trámites, accesibilidad, turismo) con control de versiones y trazabilidad.

Flujo de extremo a extremo

1. Ingreso de consulta: el usuario escribe en el canal.
2. Clasificación de intención: el Orchestrator interpreta la necesidad y estima confianza. Para ello busca contexto en la base de datos.
3. Selección y llamada a agente: enrutamiento hacia el agente más competente.
4. Respuesta del agente: el agente ejecuta RAG contra Azure AI Search y genera una respuesta fundamentada con Azure OpenAI, incluyendo evidencia y citas.
5. Devolución al Orchestrator: el agente retorna propuesta de respuesta + fuentes.
6. Unificación de personalidad: aplica el estilo institucional (tono, vocabulario, formalidad, cordialidad).
7. Guardrails y Grounding: ejecuta validaciones (moderación de contenido, políticas, límites de dominio, detección de PII, cumplimiento, respuesta en base al contenido de la base de conocimiento).
8. Actualiza el historial de conversación del usuario.
9. Respuesta al usuario: entrega una salida coherente, segura y explicable, con enlaces o referencias si corresponde.
10. Telemetría y mejora: se registran métricas (intención, rutas, latencias, fuentes citadas, feedback) para refinar índices y prompts.

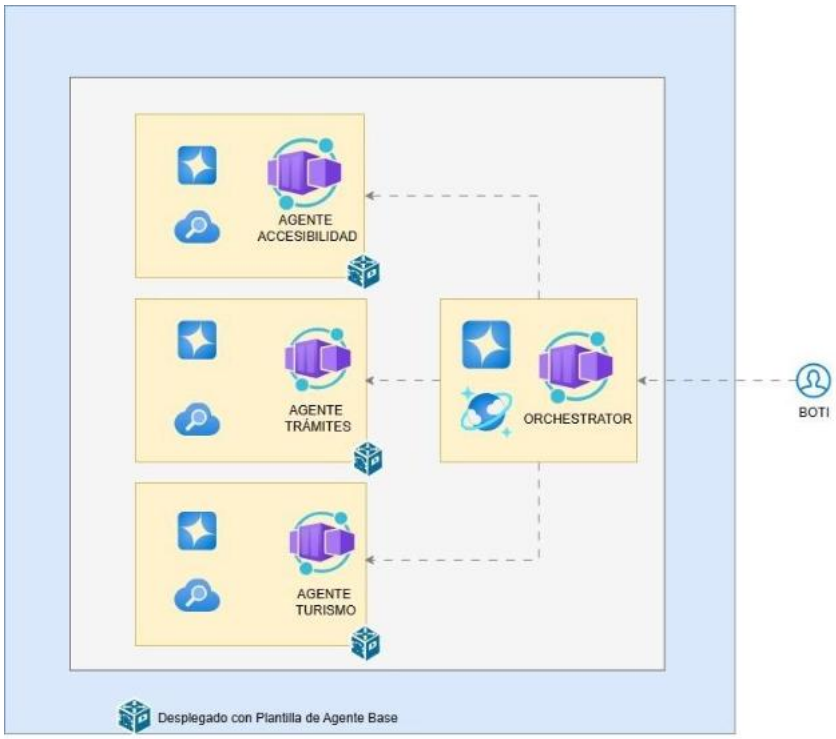


Responsabilidades clave

- Orchestrator
  - Enrutamiento por intención y confianza, fallback (p. ej., pedir aclaración o derivar a un agente generalista).
  - Gestión del historial de la conversación en la base de datos.
  - Coordinación multi-agente y resolución de conflictos (p. ej., resultados divergentes).
  - Personalidad única de cara al usuario: tono, formato, nivel de detalle, lenguaje inclusivo, consistencia terminológica.
  - Guardrails y cumplimiento: moderación, filtrado de datos sensibles, cumplimiento normativo, límites de dominio.
  - Trazabilidad y observabilidad: logs, métricas, auditorías, calidad de grounding.
- Agentes

- Especialización por dominio (Accesibilidad, Trámites, Turismo).
- RAG con Azure AI Search y modelos de Azure OpenAI.
- Formato de salida estandarizado.
- Gestión de errores del dominio y sugerencias de re-búsqueda o datos faltantes.

Todos los agentes se desplegarán a partir de una Plantilla de Agente Base (infraestructura como código), lo que asegura consistencia operativa y aceleración en la incorporación de los nuevos casos de uso. Cada agente tendrá acceso a la base de conocimiento correspondiente y a un modelo de OpenAI. **Estos recursos podrán ser únicos para todo el ambiente, sin necesidad de replicar los recursos para cada agente.**



La solución implementa una arquitectura **Multi-Agente basada en LangGraph** desarrollada en **Python** y desplegada sobre el ecosistema de **Microsoft Azure**.

El sistema se ejecuta en un entorno serverless conteinerizado, garantizando escalabilidad y modularidad. Los componentes principales de la arquitectura tecnológica son:

- **Cómputo y Despliegue:** Los desarrollos (Orquestador y Agentes) están implementados como **Azure Functions** encapsuladas dentro de **Azure Container Apps**. Esto permite ejecutar la lógica de negocio y los grafos de LangGraph en contenedores aislados y gestionados.
- **Inteligencia Artificial (LLMs):** Se utiliza **Azure AI Foundry** como plataforma unificada para la gestión, despliegue y acceso a los modelos de **Azure OpenAI** (ej. GPT-4o / GPT-3.5) que impulsan el razonamiento de los agentes.

- **Orquestación:** El flujo conversacional, el manejo de estado y el enrutamiento entre agentes se gestionan mediante el framework **LangGraph**.
- **Observabilidad:** Se utiliza **Azure Application Insights** para la recolección de métricas, trazas de ejecución, monitoreo de rendimiento y detección de errores en tiempo real.

Orquestador

Esta sección describe la arquitectura y el funcionamiento del **orquestador multiagente** basado en **LangGraph** desplegado sobre **Azure**, utilizado para canalizar las consultas de las personas usuarias hacia distintos agentes especializados (Trámites, Descubrir BA, Accesibilidad) dentro del ecosistema de "Boti".

1. Visión general de la solución

La solución implementa un **orquestador multiagente** que:

1. Recibe los mensajes del canal conversacional (por ejemplo, WhatsApp / webchat) mediante un **endpoint HTTP** en Azure.
2. Procesa cada mensaje con un **grafo de estados** (en el framework de LangGraph) que:
   - Normaliza y reformula la pregunta (nodo reformulate).
   - Clasifica la intención (nodo triage).
   - Selecciona un agente de dominio (nodo router).
   - Llama al agente (nodo agent_call), incluyendo grounding con fuentes internas vía Azure AI Search (nodo grounding).
   - Aplica guardrails (nodos blocked_list y blocked_list_agent) sobre entrada y salida.
   - Uniforma el estilo de respuesta según la personalidad de Boti (nodo personalidad).
3. Persiste estado, configuración y conversaciones en servicios de Azure para trazabilidad y mejora continua.

2. Orquestación con LangGraph

2.1 LangGraph y StateGraph

La lógica conversacional se modela con **LangGraph**, construyendo un StateGraph que define un **estado global** (AgentState) con los campos relevantes de la conversación. Además, define **nodos** (funciones async) que transforman el estado, y define **edges** condicionales que deciden el siguiente nodo según el contenido del estado.

2.2 AgentState (estado compartido)

El estado (AgentState) es un TypedDict que centraliza la información de la conversación. A grandes rasgos incluye:

- question: Última pregunta en su versión "normalizada" (tras el nodo reformulate).

- answer: Respuesta "actual", sobre la que trabaja el nodo personality para darle la voz final de Boti.
- messages: Historial de mensajes y turnos de la conversación, usado para dar contexto a los modelos.
- partial_answers: Diccionario con salidas intermedias de cada nodo.
- node_metadata: Información técnica por nodo (tiempos de ejecución, conteo de tokens, errores, etc.).
- outcome: Objeto estructurado que resume:
  - intenciones e información del triage,
  - la ruta seleccionada por el router,
  - motivos de bloqueo (guardrails),
  - cualquier dato relevante para logging y analítica.
- agents: Configuración cargada desde Cosmos: descripciones, prompts, herramientas y parámetros de cada agente de dominio.
- prompts: Plantillas de prompts globales y por nodo, también cargadas desde Cosmos.
- guardrails: Configuración de listas de palabras bloqueadas, reglas de contenido y otros filtros.

Este estado se pasa de nodo en nodo y se enriquece progresivamente.

3. Nodos del grafo y edges

A continuación, se describe el rol de cada nodo y las transiciones principales (edges) del grafo.

3.1 Nodo blocked_list

Aplica guardrails desde el primer paso, verificando si el texto de entrada del usuario contiene términos prohibidos. Si hay términos prohibidos, se marca el estado como bloqueado y se deriva al nodo undefined. Por otro lado, si no hay términos prohibidos, el input es derivado al nodo reformulate.

3.2 Nodo reformulate

Normaliza la consulta del usuario teniendo en cuenta el historial de la conversación como contexto para luego facilitar las tareas de triage, routing y grounding. Tras finalizar esta tarea, deriva el input al nodo triage.

3.3 Nodo triage

Determina el **tipo de intención** del usuario entre las siguientes opciones:

- greeting (saludo)
- about_bot (preguntas sobre el propio bot)
- out_of_scope (tema fuera del alcance de Boti)

- question_answering (consulta “normal” que puede derivar a agentes de dominio)

Si la intención detectada es "question_answering", entonces se deriva la consulta del usuario al nodo router para que seleccione al agente que responderá la pregunta. Para las demás intenciones, el input es dirigido al nodo undefined para luego ser controlado por fuera del orquestador.

3.4 Nodo router

Selecciona el **agente de dominio** más adecuado para atender la consulta. Si el nodo elige un agente válido (esto es, "accesibilidad", "tramites", "descubrir"), entonces la consulta pasa al nodo agent_call. Si el router no encuentra un agente adecuado para responder la pregunta, la deriva al nodo undefined.

3.5 Nodo blocked_list_agent

Revisa el input proveniente del router asegurando que no contenga términos prohibidos propios de la temática del agente seleccionado. Al igual que antes, si se detectan términos prohibidos, se deriva el input al nodo undefined. Si no se detectan, la consulta procede al nodo agent_call para ser respondida.

3.6 Nodo agent_call

Ejecuta el **agente especializado** seleccionado por el router. Tras obtener la respuesta la consulta, la deriva al nodo grounding.

3.7 Nodo personality

Aplica la **voz y tono** de **Boti** y reglas estilísticas acordadas con el cliente, sobre la respuesta ya técnicamente correcta. Tras finalizar esta tarea, deriva la respuesta al nodo grounding.

3.8 Nodo grounding

Normaliza y valida la respuesta del agente: revisa si contiene palabras o emojis bloqueados y si la respuesta se deriva efectivamente del contexto extraído por el nodo retrieve del agente base.

Si la respuesta contiene términos bloqueados o si no pasa el test de grounding (es decir, la respuesta no está relacionada con la información extraída por el agente base), se deriva a undefined.

Si la respuesta pasa el test de grounding, se finaliza el flujo.

3.9 Nodo undefined

Gestiona todos los casos en los que no hay una respuesta estándar de agente y deriva la consulta fuera del flujo para ser tratada por fuera del orquestador. Es el nodo terminal para estos caminos de error, bloqueo o falta de cobertura.

4. Flujo típico “end-to-end”

1. **Usuario envía mensaje**
   Llega al endpoint de Azure Functions.

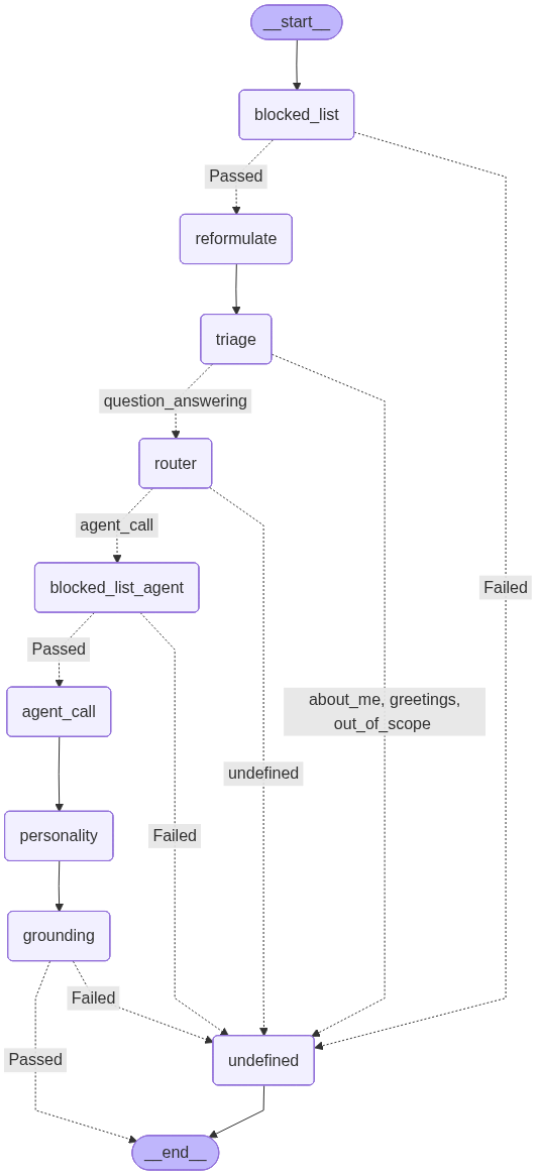
2. **Se inicializa AgentState**
   - question = mensaje original.
   - Se carga historial, agentes, prompts, guardrails desde Cosmos.
3. **blocked_list**
   - Revisión de términos prohibidos en el texto de entrada.
   - Si hay bloqueo → undefined.
   - Si no → reformulate.
4. **reformulate**
   - Normaliza la pregunta con reglas específicas (short terms, trámites, etc.).
   - Actualiza question reformulada.
   - Edge → triage.
5. **triage**
   - Clasifica intención (greeting, about_bot, out_of_scope, question_answering).
   - Si greeting/about_bot/out_of_scope → undefined.
   - Si question_answering → router.
6. **router**
   - Selecciona agente de dominio (tramites, descubrir, accesibilidad) o, si no encuentra un agente adecuado, declara route = undefined.
   - Si route = undefined → undefined.
   - Si agente válido → agent_call.
7. **blocked_list_agent**
   - Usa la ruta seleccionada (route) para aplicar una lista de palabras bloqueadas específica del agente.
   - Si hay bloqueo → undefined.
   - Si no → agent_call.
8. **agent_call**
   - Ejecuta el agente especializado seleccionado por el router, ya habiendo pasado por los guardrails específicos de blocked_list_agent.
   - Genera una respuesta técnica inicial (todavía sin voz de Boti) y la deriva al nodo personality.
9. **personality**
   - Aplica la voz y tono de Boti y formato acordado con el cliente.
   - Actualiza answer final y la deriva al nodo grounding.

10. **grounding**
    - Verifica que la respuesta sea consistente con las fuentes oficiales usando Azure AI Search (por ejemplo, verificando datos clave o consultando documentos de referencia).

- Aplica reglas de guardrails sobre la respuesta final para asegurar que la redacción final no incluya términos no permitidos.
- Si el test de grounding detecta problemas, se registra el motivo y se deriva el input al nodo undefined.
- Si el test de grounding es satisfactorio, se devuelve la respuesta al usuario.

11. **Respuesta al usuario**

- Azure Functions retorna answer al canal de origen.
- Se guarda la conversación y outcome en Cosmos.



Estructura de datos

Raíz

```
{
  "id": "string (conversation_id)",
  "history": [
    { "role": "user|assistant|system", "content": "string" }
  ],
  "conversation_data": {
    "start_date": "ISO-8601 string",
    "interactions": [ /* Interaction[] */ ]
  },

  "botmaker_id": "string (optional, tracking)",

  "_rid": "cosmos internal",
  "_self": "cosmos internal",
  "_etag": "cosmos internal",
  "_attachments": "cosmos internal",
  "_ts": 0
}
```

Donde:

- **id:** identificador único de la conversación. Se utiliza como *partition key* para recuperar, actualizar y auditar el hilo completo a lo largo del tiempo.
- **history[]:** historial en formato simple. Es la representación mínima del diálogo para reconstrucción rápida del contexto y para trazabilidad básica sin depender de métricas.
- **conversation_data.start_date:** timestamp de creación/inicio de la conversación. Permite ordenamiento temporal, y análisis por cohortes (p. ej., conversaciones iniciadas por día/semana).
- **conversation_data.interactions[]:** registro por interacción (turno) con *telemetría y snapshot*. Incluye métricas de ejecución (tiempos, modelo, tokens), datos de debugging (node_metadata, partial_answers) y el last_response como evidencia exacta de lo que se devolvió al cliente. Es la fuente principal para observabilidad, auditoría y troubleshooting.
- **botmaker_id:** identificador externo de tracking (Botmaker) asociado a la conversación o al request (según implementación). Se usa para correlacionar eventos entre plataformas y facilitar seguimiento end-to-end.
- **Metadatos internos de Cosmos (_rid, _etag, _ts, _self, etc.):** campos administrados por Cosmos DB para control de concurrencia (ETag), timestamp de última actualización, y referencias internas del documento. No forman parte del contrato funcional de la API, pero son útiles para diagnóstico y operaciones.

Interaction

```
{
  "user_id": "string",
  "user_name": "string",
  "response_time": 0.0,

  "gpt_groundedness": 0,
  "pass_rai_fairness_check": false,

  "user_ask": "string",
  "answer": "string",

  "search_query": "string",
  "intents": [],
  "detected_language": "string",

  "answer_generated_by": "string (e.g. graph_orchestrator)",
  "conversation_history_summary": "string",
  "conversation_plugin_answer": "string",

  "model": "string (e.g. gpt-5-chat)",
  "prompt_tokens": 0,
  "completion_tokens": 0,
  "prompt": "string",
  "sources": "string",
  "score": 0,
  "language": "string",

  "last_response": { /* snapshot del payload que devolviste */ },

  "node_metadata": { /* telemetría por nodo */ },
  "partial_answers": { /* salidas parciales por nodo */ }
}
```

Donde:

- **user_id / user_name**: identidad del usuario si existe; en su defecto valores por defecto (ej. “anonymous”). Se usa para trazabilidad, segmentación y auditoría.
- **response_time**: tiempo total end-to-end del turno medido por el orquestador. Sirve para performance y alertas.
- **gpt_groundedness / pass_rai_fairness_check**: flags/métricas de calidad y safety asociadas al turno. Se usan para reporting, seguimiento de compliance y monitoreo.
- **user_ask / answer**: texto original consultado por el usuario y respuesta final devuelta en ese turno. Es la representación funcional del intercambio.
- **search_query / intents / detected_language / language**: datos de clasificación/enriquecimiento (búsqueda, intención, detección/normalización de idioma). Se usan para analítica, routing y evaluación de comportamiento.
- **answer_generated_by**: origen del componente que generó la respuesta final (trazabilidad). Valores típicos esperables: graph_orchestrator (respuesta consolidada por el grafo), agent:<nombre> (respuesta generada por un agente específico), fallback/default_answer

(respuesta de degradación controlada), guardrails/blocked_list (respuesta interceptada por reglas). Sirve para medir distribución por componente y diagnosticar degradaciones.

- **model + tokens**: identificación del modelo utilizado y contadores de tokens asociados al turno para costos y observabilidad. Cuando el detalle por nodo es necesario, se complementa con los tokens/timings en node_metadata.
- **last_response**: snapshot del payload completo efectivamente devuelto por el endpoint en ese turno (incluye outcome, correlation_id, route_to/origin, etc.). Se usa para reproducibilidad, auditoría y debugging sin re-ejecutar.
- **node_metadata / partial_answers**: desglose fino del grafo por nodo: decisiones intermedias, tiempos, endpoints, prompts/tokens y salidas parciales. Es la base para explicabilidad del routing y análisis de performance/costos por etapa.

Agente Base

Esta sección describe la arquitectura y el funcionamiento del **Agente Base**, un módulo reutilizable implementado sobre **LangGraph** y **Azure OpenAI**. Este componente actúa como una plantilla agnóstica al dominio que es instanciada dinámicamente por el Orquestador para resolver consultas especializadas (como "Descubrir", "Trámites", "Accesibilidad") mediante un flujo de RAG (*Retrieval-Augmented Generation*).

1. Visión general de la solución

El Agente Base no posee conocimiento pre-cargado ("hardcodeado"). Su comportamiento se define completamente mediante **inyección de configuración** en tiempo de ejecución (prompts, schemas y herramientas).

La solución implementa un grafo de estados que:

1. **Analiza la necesidad de filtros:** Evalúa si la configuración recibida requiere extraer metadatos estructurados (ej. fechas, barrios) antes de buscar.
2. **Recupera contexto (Retrieval):** Consulta un índice específico de **Azure AI Search** mediante una búsqueda híbrida que combina vectores (embeddings) y palabras clave.
3. **Genera respuesta (Generation):** Construye una respuesta en lenguaje natural basada estrictamente en las fuentes recuperadas y las reglas de negocio definidas.

2. Orquestación con LangGraph

2.1 Definición del Workflow

La lógica conversacional se modela mediante un StateGraph compilado. El flujo se define de la siguiente manera:

- **Punto de Entrada:** Condicional (check_entrypoint).
- **Nodos:** filters, retrieve, rag_answer.
- **Transiciones (Edges):**
  - filters → retrieve
  - retrieve → rag_answer
  - rag_answer → END

2.2 AgentState (Estado Compartido)

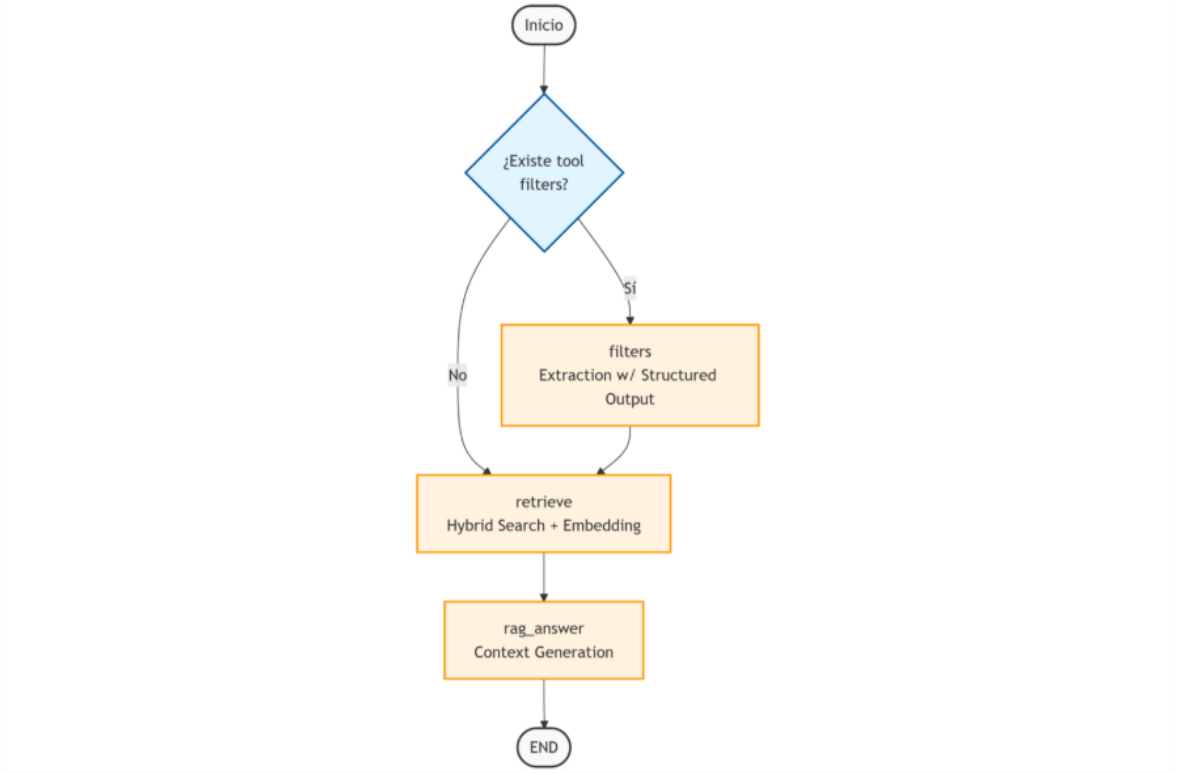
El estado (AgentState) es un diccionario tipado (TypedDict) que centraliza toda la información durante el ciclo de vida de la ejecución. Su estructura es:

- **question** *(str)*: La consulta del usuario que se está procesando (puede ser reformulada).
- **original_question** *(str)*: La consulta original intacta, conservada para referencia.
- **answer** *(str)*: La respuesta final generada por el LLM.
- **tools** *(list)*: Lista de configuraciones inyectadas desde el request (ej. configuración del índice, schemas de filtros).
- **prompts** *(dict)*: Diccionario con las plantillas de sistema (*System Prompts*) para generación y extracción.
- **context** *(str)*: Texto consolidado de los documentos recuperados por el nodo retrieve.

- **filters** *(str | None)*: Estructura con los filtros extraídos (ej. fechas, ubicación) para aplicar en la búsqueda. Es None si no aplica.
- **node_metadata** *(dict)*: Métricas técnicas por nodo (tiempos, tokens, etc.).
- **partial_answers** *(dict)*: Almacén de salidas intermedias de los nodos.

3. Descripción de Nodos y Lógica

3.0 Gráfico de nodos



3.1 Conditional Entry Point: check_entrypoint

Es la primera lógica que se ejecuta al instanciar el agente.

- **Función:** Verifica la configuración en state["tools"].
- **Lógica de decisión:**
  - Si existe una tool de tipo **filters**, el flujo va al nodo **filters**.
  - Si **no** existen filtros, el flujo salta directamente al nodo **retrieve**.

3.2 Nodo filters (Extracción Estructurada)

Este nodo transforma el lenguaje natural en parámetros de búsqueda estructurados.

- **Input:** question y el schema JSON definido en tools.
- **Proceso:** Utiliza un LLM con capacidad de *Structured Output* para instanciar un modelo Pydantic dinámico basado en el schema.

- **Output:** Actualiza state["filters"] con valores normalizados (ej. {"fecha_inicio": "2025-05-11", "barrio": "Villa Crespo"}).
- **Transición:** Hacia el nodo retrieve.

3.3 Nodo retrieve (Búsqueda Híbrida)

Encargado de la recuperación de información oficial en Azure AI Search.

- **Input:** question, filters y configuración del índice (index_name).
- **Proceso:**
  - **Generación de Embeddings:** Vectoriza la question actual utilizando el modelo de embeddings configurado.
  - **Búsqueda Híbrida:** Ejecuta una consulta que combina:
    - Búsqueda vectorial (similitud semántica).
    - Búsqueda por palabras clave (BM25).
  - **Filtrado:** Aplica los filtros OData derivados de state["filters"] (si existen) para restringir los resultados.
- **Output:** Concatena los fragmentos más relevantes y actualiza state["context"].
- **Transición:** Hacia el nodo rag_answer.

3.4 Nodo rag_answer (Generación)

Genera la respuesta final al usuario.

- **Input:** context, question, prompts.
- **Proceso:**
  - El LLM asume el rol definido en el prompt del sistema.
  - Aplica reglas de *Grounding*: la respuesta debe basarse únicamente en context.
  - Si el contexto está vacío o no es relevante según los filtros, genera una respuesta indicando la falta de información.
- **Output:** Actualiza state["answer"].
- **Transición:** END.

4. Flujo de Datos Típico (Ejemplo)

1. **Input:** El orquestador envía la pregunta *"¿Qué hay para hacer el 11 de mayo en Villa Crespo?"* y la configuración del agente "Descubrir".
2. **Check:** Se detecta configuración de filtros → Nodo filters.
3. **Filters:** Se extrae fecha="2025-05-11" y barrio="Villa Crespo".
4. **Retrieve:**

a. Se crea el vector de la pregunta.

b. Se busca en el índice "descubrir" usando el vector + texto, filtrando por fecha y barrio.

c. Se recuperan eventos relevantes.

5. **Rag_answer:** El LLM redacta la respuesta usando la información de los eventos recuperados.

6. **Output:** Se devuelve la respuesta final al orquestador.

Ingesta de la Base de Conocimientos

A continuación, se enumeran los diferentes orígenes de datos utilizados para la ejecución de este proyecto:

El sistema utiliza servicios de datos gestionados en Azure para persistencia y recuperación de información:

1. **Azure AI Search (Búsqueda Vectorial e Híbrida):**
    - Actúa como el motor de conocimiento para los agentes. Almacena los índices vectoriales de la información oficial (ej. trámites, eventos).
    - Permite al **Agente Base** realizar búsquedas híbridas (vectores + palabras clave) para el proceso de RAG (Retrieval-Augmented Generation).
2. **Azure Cosmos DB (Persistencia y Configuración):**
    - **Recursos del Sistema:** Almacena la configuración dinámica de los agentes, prompts del sistema, schemas de herramientas y reglas de guardrails.
    - **Conversaciones:** Persiste el historial de chat y el estado (AgentState) de cada interacción, permitiendo mantener el contexto y la trazabilidad de las sesiones.
3. **CMS Drupal (Gobierno de la Ciudad):** Fuente primaria de información no estructurada. Se consumen los **endpoints JSON** públicos de CABA que contienen los detalles de trámites, servicios y eventos.

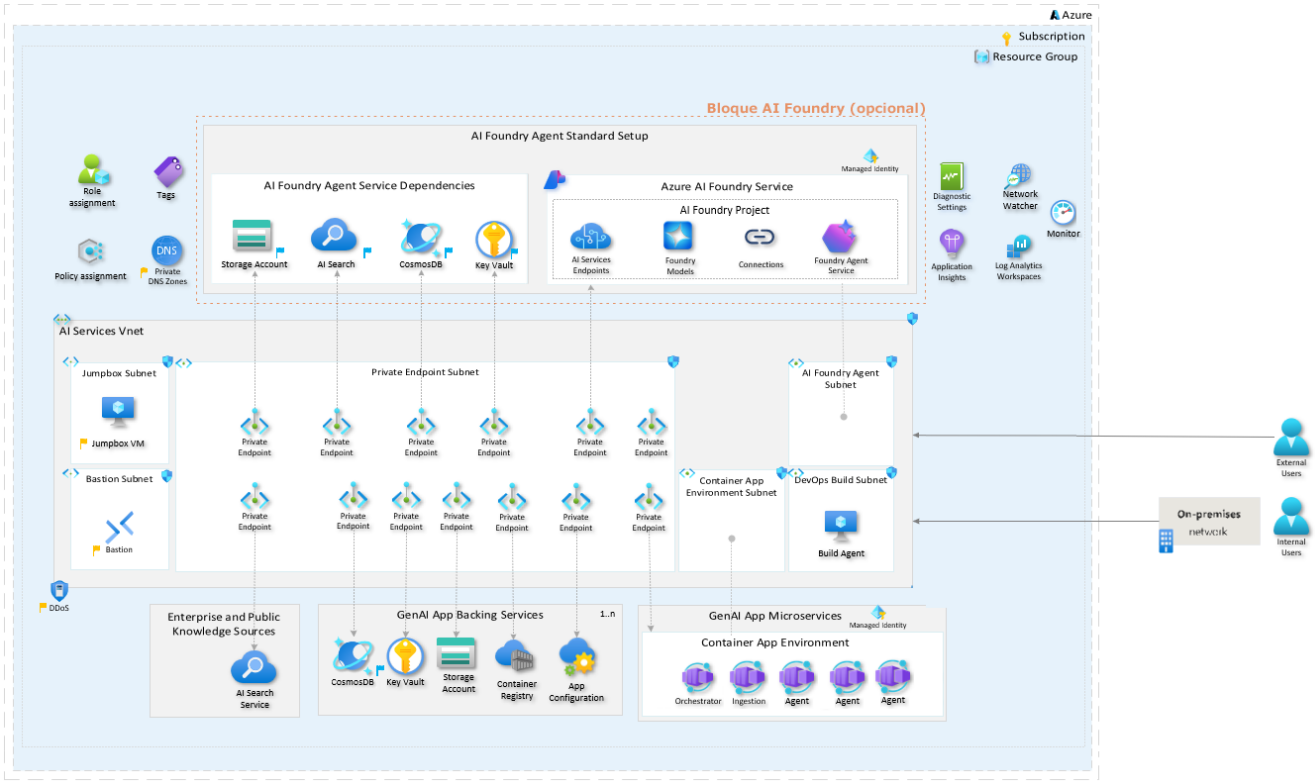
Para mantener la base de conocimiento actualizada, se desarrolló un **proceso de Ingesta automatizado** que corre como una tarea programada (job) dentro de **Azure Container Apps**. Este proceso se ejecuta todas las noches y sigue una lógica de **Sincronización Incremental** para optimizar el consumo de recursos y tokens.

El flujo de trabajo (Pipeline) realiza los siguientes pasos:

- **Extracción (Extraction):**
    - El servicio se conecta a los endpoints de **Drupal** del GCABA.
    - Descarga la información cruda en formato JSON correspondiente a cada temática (Agente).
- **Detección de Cambios (Change Data Capture):**
    - Se aplica una función de **hashing** sobre el contenido descargado.
    - El sistema compara estos hashes con los metadatos almacenados actualmente en **Azure AI Search**.
    - **Lógica de decisión:**
        - Si el documento no existe en el índice: Se marca para **Alta**.
        - Si el documento ya no existe en el origen pero sí en el índice: Se marca para **Baja (Borrado físico)**.
        - Si el hash coincide: Se ignora (sin cambios).
- **Transformación y Enriquecimiento (Transformation):**
    - **Normalización:** Los documentos nuevos o modificados se transforman al formato estándar requerido por el Agente Base.
    - **Resumen con IA:** Se utiliza un **LLM** para generar un resumen conciso del trámite o servicio, facilitando la comprensión semántica.

- **Carga e Indexado (Load):**
  - **Vectorización:** Se generan los embeddings (vectores) del contenido textual y del resumen.
  - **Indexación:** Se suben los documentos procesados (texto + vectores + metadatos) al índice correspondiente en **Azure AI Search**, asegurando que los agentes tengan acceso inmediato a la información renovada.

Arquitectura



Descripción

Bloque GenAI Microservices

Propósito: Alojar los microservicios de experiencia conversacional y decisión. Orchestrator, agentes de dominio (p. ej., Accesibilidad, Trámites, Turismo) y utilitarios comunes.

Componentes:

- Azure Container Apps dentro de un Container Apps Environment.
- Servicios/microservicios:
  - Orchestrator API: enrutamiento por intención, coordinación multi-agente, fusión de resultados, unificación de personalidad, aplicación final de guardrails.
  - Agentes de dominio (p. ej., Accesibilidad / Trámites / Turismo): lógica de dominio, consultas RAG a Azure AI Search, generación controlada con Azure OpenAI, formateo de evidencia.

- Agente de Ingesta de información en Azure AI Search.
- Integraciones: autenticación con Microsoft Entra ID (ex Azure AD), telemetría con Azure Monitor / App Insights, secretos vía Managed Identity + Key Vault.

Bloque GenAI Backing Services

Propósito: Servicios de soporte transversales a los microservicios.

Componentes:

- Azure Cosmos DB: estado conversacional, historiales, etc.
- Azure Storage Account (Blob): adjuntos, export de telemetría, etc.
- Azure Key Vault: secretos, certificados y claves de conexión.
- Azure Container Registry (ACR) (si aplica): imágenes de microservicios.

Bloque AI Foundry (opcional)

Propósito: Estandarizar experimentación, evaluación y gobierno de activos de IA (prompts, flujos, conexiones, modelos y agentes) y acelerar el ciclo del laboratorio a producción.

Componentes:

- Azure AI Foundry Project (antes AI Studio):
- Prompt Flow / Flows: orquestación de pipelines de RAG y evaluaciones.
- Data / Connections: fuentes de datos aprobadas para experimentos.
- Model Catalog & Deployments: selección y despliegue de modelos (Azure OpenAI, modelos OSS).
- Agent Service: packaging/hosting de agentes definidos en Foundry (si se usa).
- Dependencias estándar: Storage, Key Vault, Container Registry, App Insights.
- Integración CI/CD: Azure DevOps (o GitHub) con Build Agent y políticas de aprobación.

Bloque Private Endpoints (Red y Seguridad)

Propósito: Aislar el tráfico, eliminar exposición pública de PaaS y controlar la resolución DNS.

Componentes:

- VNet “AI Services VNet” con subnets dedicadas
- Private Endpoints a: Azure OpenAI, Azure AI Search, Storage, Key Vault, Cosmos DB, App Config, etc.
- Private DNS Zones enlazadas a la VNet para cada recurso con PE.

Bloque AI Search Service

Propósito: Servicio de búsqueda vectorial para RAG.

Componentes:

- Azure AI Search con índices vectoriales por dominio (p. ej., Accesibilidad, Trámites, Turismo).
- Skillsets/enrichment (si aplica) para extracción de entidades, normalización y chunking.
- Data Sources: conectores a repos corporativos (SharePoint, DBs, Storage) y fuentes públicas curadas.
- Embeddings: generadas con Azure OpenAI (o equivalentes) durante ingesta/actualización.

Seguridad

El modelo de seguridad del proyecto se estructura en capas, protegiendo tanto el acceso a la infraestructura como la integridad de las conversaciones.

1. Seguridad de Red e Infraestructura (Azure APIM): El acceso a los servicios del orquestador no se realiza de forma directa. Se implementa una capa de gestión y protección mediante **Azure API Management (APIM)**, que actúa como puerta de enlace única.

- **Control de Acceso (IP Whitelisting):** Se configuran políticas estrictas de restricción de IPs. Solo las direcciones IP autorizadas en la **lista blanca** (whitelisted IPs) pueden establecer conexión con los endpoints del sistema, rechazando cualquier tráfico no autorizado en el perímetro.

2. Seguridad Aplicativa (Guardrails y Moderación): Dentro del flujo conversacional, se aplican barreras de protección (Guardrails) para moderar el contenido:

- **Listas de Bloqueo (Entrada):** Se verifica el texto ingresado por el usuario contra una *blocked_list* global. Si contiene términos prohibidos, el flujo se desvía inmediatamente.

Pruebas

En la anterior iteración de los agentes de forma individual, la metodología de testeo que se realizó fue de "caja negra".

Con esta forma de caja negra, se introducía la consulta del usuario (input), lo que ocurría dentro de la caja no era parte del objeto de estudio, y se obtenía la respuesta (output), y en base a una respuesta esperada se hacían las evaluaciones pertinentes,

Con el agregado del orquestador y el aumento del alcance de la solución, esta forma de realizar las pruebas no se desechó por completo, pero si es insuficiente para cubrir todas las partes de la solución.

Por eso se realizó un cambio en la metodología, para lograr cubrir todas las necesidades.

En esta segunda etapa de Boti como agente conversacional en la cual se agregó el Orquestador, en la cual se pueden sumar o quitar agentes (trámites, accesibilidad, etc.), se decidió realizar ciclos de prueba sobre cada uno de los nodos del orquestador, sumado a las pruebas de pregunta-respuesta sobre los distintos agentes.

Herramientas

Se utilizó el lenguaje **Python**, para realizar una serie de scripts. Estos scripts se encargaban de:

- Leer los datasets (lista de preguntas – respuestas esperadas) provistos por el cliente.
- Enviar estas preguntas al endpoint de la solución y obtener las respuestas.
- Capturar estas respuestas, guardarlas y generar un archivo .json con todas las interacciones.
- Consumir este archivo para realizar las pruebas y obtener métricas.

Utilización de la librería **Ragas** (https://docs.ragas.io/en/latest/) para obtención de métricas evaluativas de LLMs.

Uso de **Postman** para evaluación rápida y pruebas manuales al comienzo del desarrollo.

Las pruebas de carga y stress fueron realizadas con **JMeter**.

Alcance

Las pruebas informadas en este documento fueron exclusivas de los agentes Trámites y Accesibilidad, ya que estos eran los más robustos en términos de desarrollo y los que tenían mejor calidad de la información.

Sobre el agente Descubrir se realizaron pruebas luego de la salida de producción, que coincide aproximadamente con el periodo de finalización del desarrollo de este.

Tipos de pruebas

Funcionales

- Cuantitativas

Este tipo de pruebas arrojan un resultado certero y directo, proveniente de una comparación directa entre lo que se espera de la prueba y el resultado de la misma.

Ejemplos: pruebas sobre nodos *triage* o *router*.

- Cualitativas

Pruebas que son apreciativas de la calidad, pero que no hay una comparación directa con un resultado certero.

Ejemplos: pruebas sobre nodos *reformulate* o *personality*.

- Otras

Pruebas que arrojan un resultado binario, en donde los casos negativos/falsos son analizados.

Ejemplo: pruebas sobre el *blocked_list* o *grounding*.

No Funcionales

- Consumo

Estudio sobre el consumo de tokens de las interacciones de los usuarios.

- Tiempos promedio de respuestas

Tiempos promedio para respuestas para pruebas normales.

- Pruebas de Carga

Pruebas realizadas para calcular la cantidad de usuarios capaces de interactuar con la solución al mismo tiempo manteniendo un rendimiento dentro de los estándares aceptables.

- Pruebas de Stress

Pruebas para detectar el punto de ruptura de la aplicación, y estudiar si es capaz de recuperarse luego de la prueba.

Pruebas sobre nodos

- Nodos *blocked_list* y *blocked_list_agent*.

Efectividad: 100%

Se envía una serie de consultas que contenga palabras de la lista de palabras bloqueadas otorgadas por el cliente, y se verifica que bloquee dichas palabras.

Estos nodos tienen un funcionamiento mecánico y no dependen de un LLM. Por lo que por código detiene inmediatamente las consultas, originales o reformuladas, que contengan una de estas palabras.

- Nodo *reformulate*.

Prueba cualitativa.

Evaluación: 4.1 promedio

Este nodo se testeo con la ayuda de un LLM y un prompt preparado para evaluar similitud semántica entre los distintos inputs y el output generado, así generando un puntaje cualitativo.

Este prompt se le pasa al LLM el cual genera una graduación del 1 al 5, siendo un 1 algo totalmente alejado de la intención del usuario y 5 un mensaje idéntico o casi idéntico en términos semánticos.

- Nodo *triage*.

Prueba cuantitativa

Efectividad: 81%, en base al dataset de accesiblidad y trámites.proporcionado por el cliente.

Se compara el resultado obtenido de la prueba, con el brindado en el dataset y se verifica con una igualdad directa, siendo una prueba positiva que los resultados sean idénticos (que ambos sean caso de "question_answering" para una intención de consultar por algo del usuario, por ejemplo).

- Nodo *router*.

Prueba Cuantitativa

Efectividad: 95%, en base al dataset de accesiblidad y trámites. proporcionado por el cliente.

Se compara el resultado obtenido de la prueba, con el brindado en el dataset y se verifica con una igualdad directa, siendo una prueba positiva que los resultados sean idénticos (que el orquestador derive al agente "tramites" si la consulta está en el apartado de trámites en el dataset provisto por el cliente, por ejemplo).

- Nodo *personality*.

Prueba cualitativa.

No se realizaron pruebas propiamente sobre este nodo. Si se realizaron pruebas de similitud semántica con la librería Ragas, para evaluar que tan similar es la respuesta obtenida en la interacción con el agente contra la respuesta esperada del dataset provisto por el cliente, arrojando un 79.5% de similitud promedio entre las respuestas del dataset y las respuestas obtenidas de las interacciones con el asistente.

En estas pruebas, un resultado que arroje porcentajes superiores al 75-80% indican un buen grado de similitud entre dos oraciones, siendo resultados superiores a 90% dos oraciones con palabras casi idénticas.

- Nodo *grounding.*

Efectividad: 98%

Se filtra las consultas que el resultado fuera falso, para evaluarlas manualmente y corroborar con las fuentes obtenidas por el asistente.

Las consultas que den un resultado positivo para este nodo también son evaluadas para corroborar su veracidad utilizando las métricas provistas por Ragas como *faithfulness, context_precision* y *context_recall* (todas métricas que, si la respuesta está basada en datos correctos traídos por el agente, van a dar resultados altos).

Pruebas No Funcionales

- Consumo

Promedio: 8250 input / 377 output

Medición sobre el consumo de tokens totales para generar una respuesta.

- Tiempos

Promedio: 8.6 segundos

Tiempo de respuesta promedio sobre 330 consultas dentro de las consultas de accesiblidad y trámites para pruebas de tipo secuenciales.

- Pruebas de carga y pruebas de stress

Estas pruebas se realizaron simulando la entrada masiva de conversaciones de usuarios de forma simultánea al agente utilizando el dataset provisto por el cliente.

Las pruebas fueron iterativas, de una menor cantidad de usuarios a una mayor cantidad. Estos usuarios simulados se midieron en un periodo de tiempo de 1minuto.

El primer objetivo de esta prueba y todas sus iteraciones fue el de alcanzar la cantidad estimada por el cliente:

Conservador (probable en early-prod)

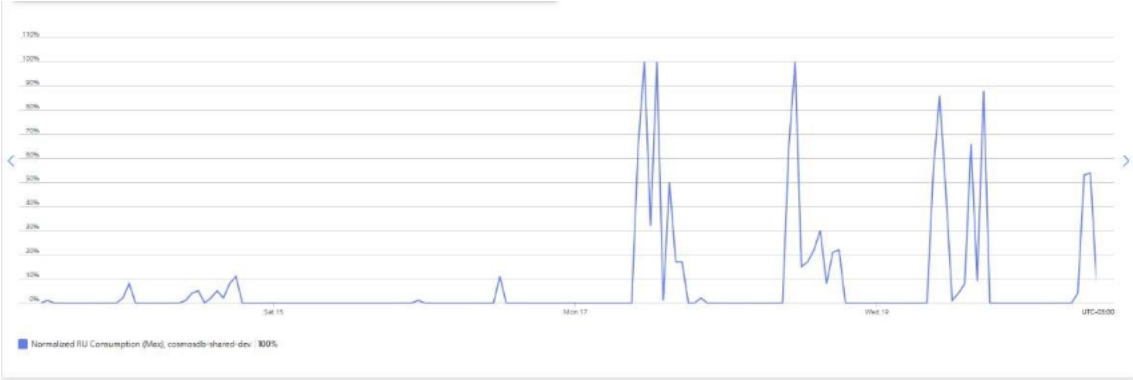
- Orquestador: **2.000 consultas/min → 33.3 req/s**
- Por minuto: 2.000; por hora pico: 120.000

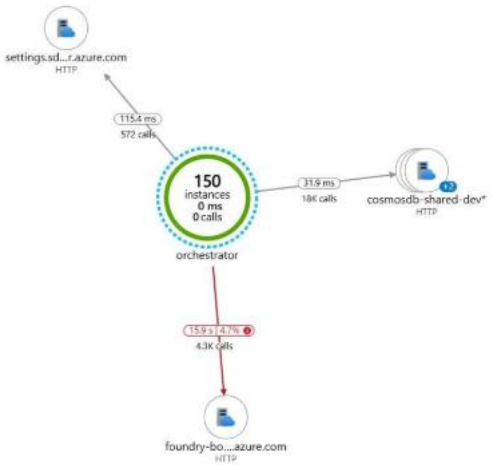
Estas iteraciones de prueba llevaron a múltiples cambios en la arquitectura del proyecto debido a los requerimientos y la necesidad de escalar para soportar la carga deseable.

<u>Base de datos:</u>

Durante el proceso de pruebas encontramos que la base de datos llegaba al 100% en cargas de 500 – 600 usuarios por segundo.

Por lo que se procedió al aumento de la capacidad de RUs.





Este cambió llevó a mejoras en los tiempos de cómputo de la base de datos y así, mejorando los tiempos de respuesta.

Réplicas:

En la configuración por default, teníamos una cantidad de 10 réplicas, estas capaces de soportar hasta 10 peticiones.

En las primeras pruebas se detectó que esto era un limitante para la cantidad de peticiones que se esperaba, por lo que se decidió aumentar la cantidad de réplicas a 80 y disminuir la cantidad de *requests* por réplica a 5, esto conduce a las réplicas no se saturen con demasiadas solicitudes y que la activación de cada una sea más rápida.

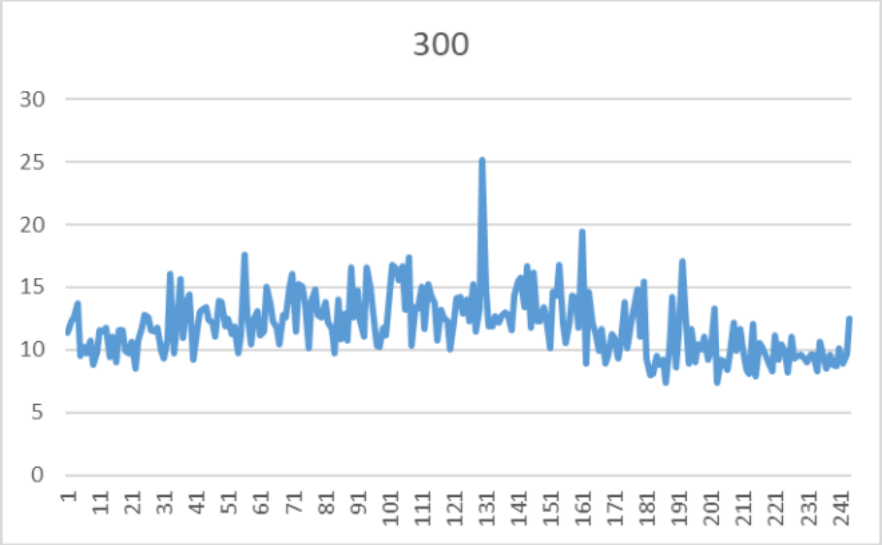
Aumento de la capacidad de cómputo

Se realizó un aumento en la CPU y la RAM del Orquestador para que pueda mantener que los *containers* sean capaces de mantener el aumento de réplicas funcionando.

Performance

Actualmente, el entorno productivo tiene una capacidad de carga de 300 usuarios por minuto donde el promedio se encuentra en 11.89 segundos, tiempo similar al que se encuentra el usuario normalmente.

Usuarios / Minuto	Tiempo (promedio)
300	11.89 seg
400	16.96 seg
500	24.52 seg



Es un área en la que se continúa trabajando, ayudándonos de las mediciones de usuarios reales que obtenemos del monitoreo del orquestador y los agentes.

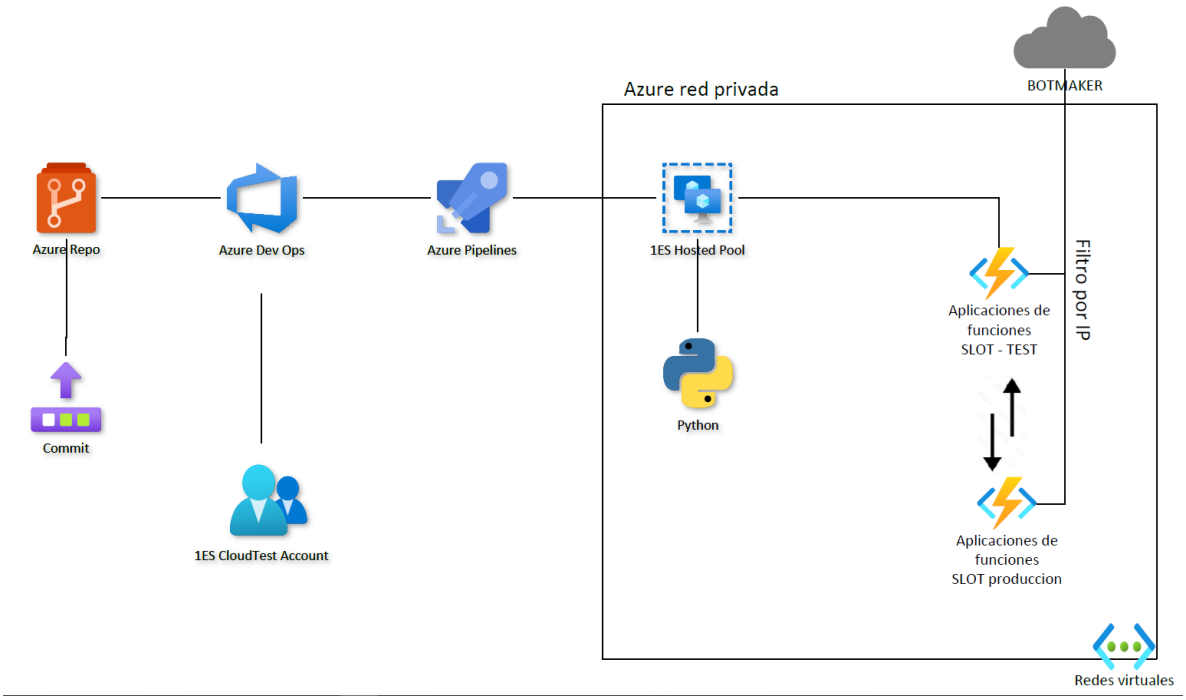
Despliegue

El despliegue de la solución se realiza de manera automatizada utilizando pipelines de **CI/CD (Integración y Despliegue Continuo)** en **Azure DevOps**, asegurando un ciclo de vida del software ágil y controlado.

El proceso de despliegue sigue el siguiente flujo:

1. **Construcción (Build):** Al confirmar cambios en el repositorio, el pipeline se dispara para compilar el código y construir las imágenes Docker de los servicios (Orquestador, Agentes o Ingesta).
2. **Registro (Push):** Las imágenes generadas se etiquetan y se suben al **Azure Container Registry (ACR)**, que actúa como el repositorio privado de imágenes del proyecto.
3. **Despliegue (Deploy):** El pipeline instruye a **Azure Container Apps** para que descargue la nueva versión de la imagen desde el ACR y actualice los contenedores en ejecución. Esto

permite realizar actualizaciones (revisiones) rápidas y gestionar el tráfico entre versiones si fuera necesario.



Monitoreo

El monitoreo de una solución en producción es clave para asegurar que el sistema funcione de manera estable y confiable a lo largo del tiempo. El análisis continuo de métricas, registros y señales operativas permite detectar de forma anticipada posibles problemas de rendimiento, errores recurrentes o comportamientos inesperados que podrían impactar tanto en la experiencia del usuario como en el cumplimiento de los objetivos del servicio.

En soluciones basadas en inteligencia artificial y arquitecturas en la nube, este proceso cobra especial importancia. El monitoreo no solo permite verificar la disponibilidad y el rendimiento de la infraestructura, sino también entender cómo interactúan los distintos componentes del sistema, evaluar el uso de los recursos y asegurar la estabilidad del flujo completo de procesamiento en un entorno productivo.

Por último, este análisis también aporta una visión clara del negocio y de las necesidades reales de los usuarios que interactúan con los distintos agentes. Esta información resulta clave para orientar decisiones futuras, especialmente en lo relacionado con el escalamiento de la solución y la mejora continua de la calidad del servicio.

Conversaciones

Desde la salida a producción del 1/12/2025 hasta el 9/12/2025, el orquestador y los agentes recibieron un 10% del total de los posibles usuarios. Durante estos nueve días se observó la siguiente distribución de consultas:

En donde se observa que la gran mayoría son consultas que el nodo *triage* determina que no están contempladas dentro del alcance de los tres agentes actualmente disponibles.

De las consultas que continúan con el flujo de la conversación, se observa un gran volumen de consulta dirigidas al agente de Trámites.

A partir del día 10/12/2025, se aumentó la cantidad de usuarios desde un 10% a un 25% en donde se observa una distribución similar en términos porcentuales de conversaciones realizadas hacia los agentes (hasta el 15/12/2025).

Performance

El análisis de los tiempos de respuesta de las conversaciones derivadas a agentes muestra una estabilidad: la mayoría de las interacciones se resuelven en un intervalo comprendido entre 10 y 15 segundos. Este comportamiento sugiere que el sistema mantiene un nivel de servicio consistente para la mayoría de los usuarios.

Examinando los percentiles estadísticos, se observa que el tiempo de respuesta p95, calculado sobre 1.250 muestras, es de 15 segundos, mientras que el p99, basado en 363 muestras, alcanza los 22 segundos, en el contexto de un total de 14.730 consultas gestionadas.

Las conversaciones que se sitúan en el percentil p99 presentan, en su mayor parte, tiempos de espera de entre 25 y 35 segundos. Estos casos corresponden a situaciones donde se ha identificado la causa del problema y se está trabajando en una solución para evitar que se repitan a futuro.

La principal causa de estas demoras es que, en el primer intento, el orquestador no consigue respuesta por parte del agente y necesita realizar uno o incluso dos intentos adicionales para obtener la información solicitada. Aunque el usuario final acaba recibiendo la respuesta, el proceso implica un tiempo de espera más prolongado.

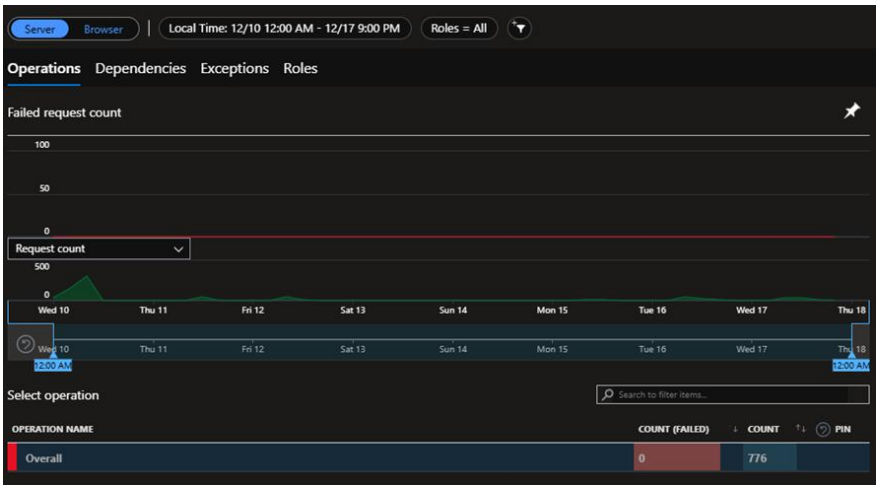
Por otro lado, los tiempos de respuesta de los agentes muestran un pico en 2,1 segundos, con un p95 de 5,8 segundos (554 muestras) y un p99 de 21 segundos (122 muestras), sobre el total de 14.730 consultas.

Este análisis permite identificar oportunidades de mejora en la gestión de los agentes y el orquestador, optimizando la experiencia del usuario y la eficiencia operativa del sistema.

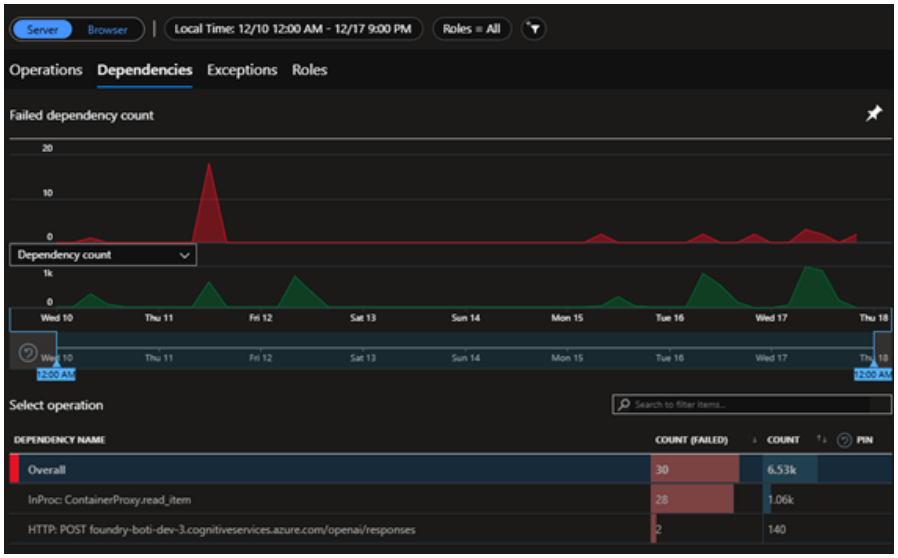
Failures

En este apartado de Azure podemos visibilizar las respuestas en la que estamos teniendo algún tipo de falla.

En el periodo comprendido entre el 10/12/2025 hasta el 17/12/2025 no contamos con fallas significativas en las respuestas

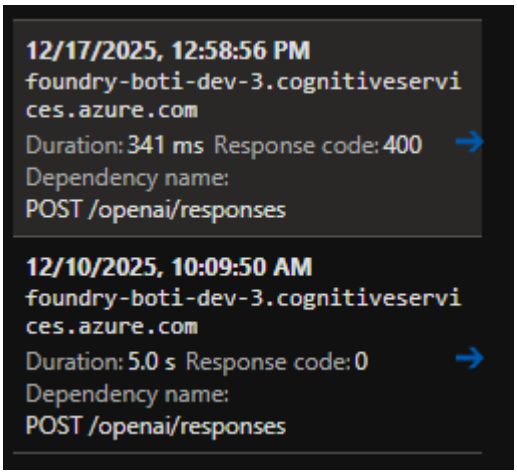


Si expandemos hacia la pestaña de Dependencias, podemos observar algunos errores 0 y un solo caso de un error 400.



Haciendo foco primero en las respuestas tipo **0**, la gran mayoría están asociados a una posible falla en alguna prueba realizada por alguno de los equipos encargados de la prueba, debido a una mala configuración del *body* de la petición, al dejarlo vacío.

Los otros errores posiblemente estén asociados una falla de comunicación entre el Orquestador y los Agentes, que luego es solucionado por el *retry* que vuelve a ejecutar la llamada.

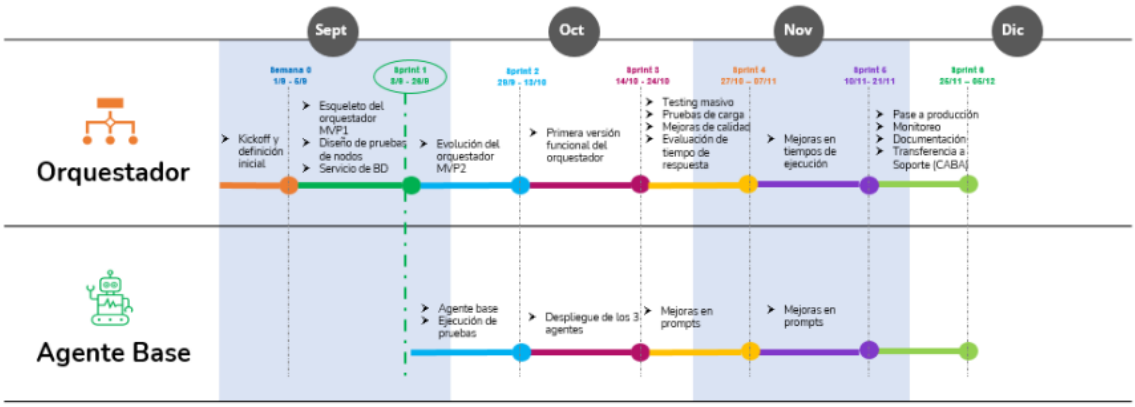


Gestión del proyecto

La gestión del proyecto tuvo como objetivo asegurar la entrega progresiva del orquestador y del agente base, reduciendo riesgos de implementación y alineando en todo momento el alcance con las prioridades definidas por CABA. Para ello se adoptó un enfoque ágil, trabajando en sprints de dos semanas de duración con una cadencia fija de planificación, seguimiento y revisión de avances.

1. Roadmap

Se definió un roadmap de alto nivel con hitos claros: preparación de la infraestructura, construcción del orquestador, implementación del agente base y de los primeros agentes funcionales, ejecución de pruebas (funcionales, masivas y de carga), mejoras y refinamiento y salida a producción. Sobre este roadmap se gestionó el backlog de trabajo en Jira como herramienta de gestión (épicas, historias y tareas), permitiendo priorizar el valor de negocio y dar visibilidad al estado del proyecto.

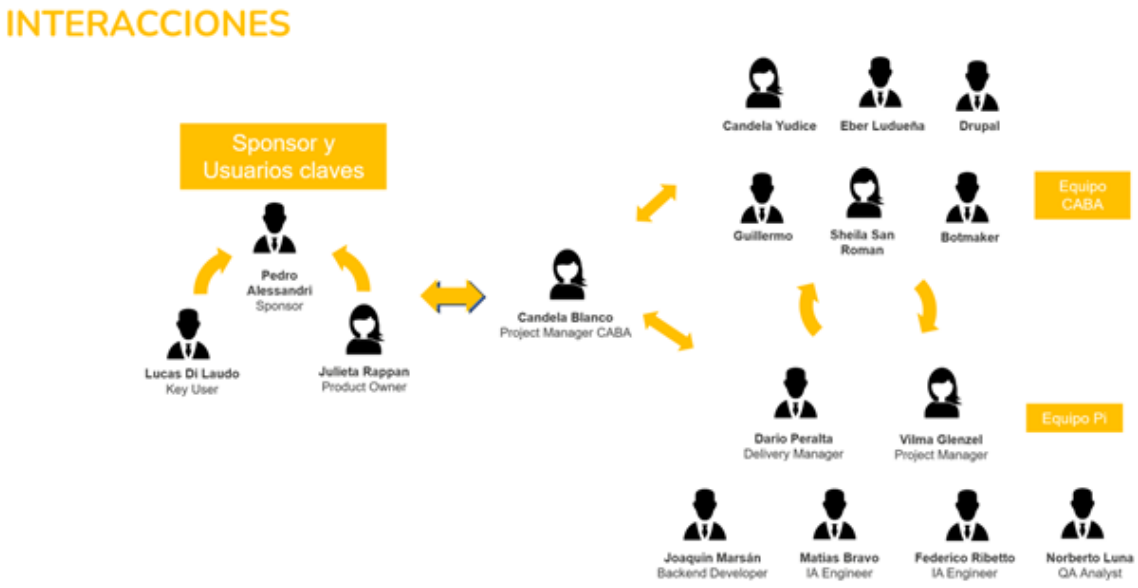


Roadmap de implementación del orquestador y agente base.

2. Organización del equipo y gobernanza

- Equipo

El equipo se organizó en frentes de trabajo complementarios (arquitectura e infraestructura, datos, orquestador, agentes, pruebas y monitoreo), coordinados por el Delivery Manager y el Project Manager. Se definieron roles y responsabilidades para cada actor (equipo técnico, referentes de CABA y terceros involucrados), asegurando puntos de contacto claros y una toma de decisiones ágil.



Estructura de roles y frentes de trabajo

- Modelo de trabajo y comunicación

En términos de gobernanza y comunicación, se estableció una cadencia de reuniones que incluyó dailies técnicas, espacios de coordinación entre frentes y sprints reviews con el cliente para revisión de avances y toma de decisiones. Se documentaron acuerdos, decisiones de diseño y cambios relevantes, garantizando trazabilidad y alineamiento entre todas las partes (equipo interno, CABA y terceros).

> Modelo de comunicación
>
> - Cadencia: dailies técnicas internas y con CABA, coordinación semanal interna, review quincenal con CABA, sincronización permanente entre PMs y Delivery Manager.
> - Objetivo de cada espacio: dailies (desbloqueo operativo), coordinación (alinear frentes y riesgos), review (decisiones de alcance, priorización y roadmap).
> - Principio de gestión: decisiones basadas en impacto en entregable, riesgo y esfuerzo.
>
> - Gestión del alcance, riesgos y cambios

Desde la gestión se realizó un control activo del alcance. El alcance inicial se centró en el orquestador y en un conjunto acotado de agentes: Tramites, Accesibilidad y Turismo, sobre el cual se incorporaron ajustes a través de un proceso formal de gestión de cambios, evaluando impacto en tiempos, esfuerzo y riesgos antes de su aprobación e incorporación al plan.

Se mantuvo una gestión sistemática de riesgos y dependencias, identificando riesgos técnicos, funcionales y de coordinación (por ejemplo, definiciones pendientes, cambios en integraciones o ventanas de despliegue). Para cada riesgo se definieron acciones de mitigación, responsables y

mecanismos de seguimiento, escalando oportunamente cuando fue necesario ajustar alcance, plazos o prioridades.

3. Coordinación de pruebas, despliegues y pases de ambiente

La coordinación de pruebas y pases de ambiente se gestionó como parte central del plan del proyecto. Se acordaron criterios de aceptación, se organizó la ejecución de pruebas funcionales y masivas en ambientes controlados, y se planificaron los despliegues a producción escalonados y estrategia de rollback.

Finalmente, se puso foco en la documentación, transferencia y cierre. Se consolidaron los principales artefactos técnicos y operativos (diagramas, lineamientos para incorporación de nuevos agentes, pautas de monitoreo y operación), y se propuso instancia de handover hacia los equipos responsables de la evolución y soporte de la solución. Como parte del cierre se relevaron lecciones aprendidas, identificando prácticas a mantener y oportunidades de mejora para futuros proyectos similares.

Esta gestión integral permitió mantener control sobre alcance, plazos y riesgos, habilitando una salida productiva ordenada y una base sólida para la evolución posterior del orquestador y del agente base.

Recomendaciones y mantenimiento del modelo

Para garantizar la operatividad y calidad del servicio a largo plazo, se recomiendan las siguientes acciones de mantenimiento:

- **Monitoreo de Calidad de Respuestas:** Revisar periódicamente una muestra de las interacciones almacenadas en Cosmos DB para detectar alucinaciones o respuestas imprecisas. Esto servirá de input para ajustar los *prompts* del sistema.
- **Escalabilidad de Infraestructura:** Aunque Azure Container Apps escala automáticamente hasta cierto punto, se debe monitorear el consumo de recursos en *Application Insights*. Si el caudal de usuarios aumenta drásticamente, puede ser necesario ajustar los límites de réplicas (mínimos y máximos) de los contenedores para evitar latencia.
- **Supervisión del Pipeline de Ingesta:** Verificar diariamente que el *job* nocturno de sincronización con Drupal se haya ejecutado sin errores para asegurar que la información no esté obsoleta.